

MOQ

Topic

- TDD
- Introduction to MOQ
- Scenarios
- Fakes
- Stub
- MOQ

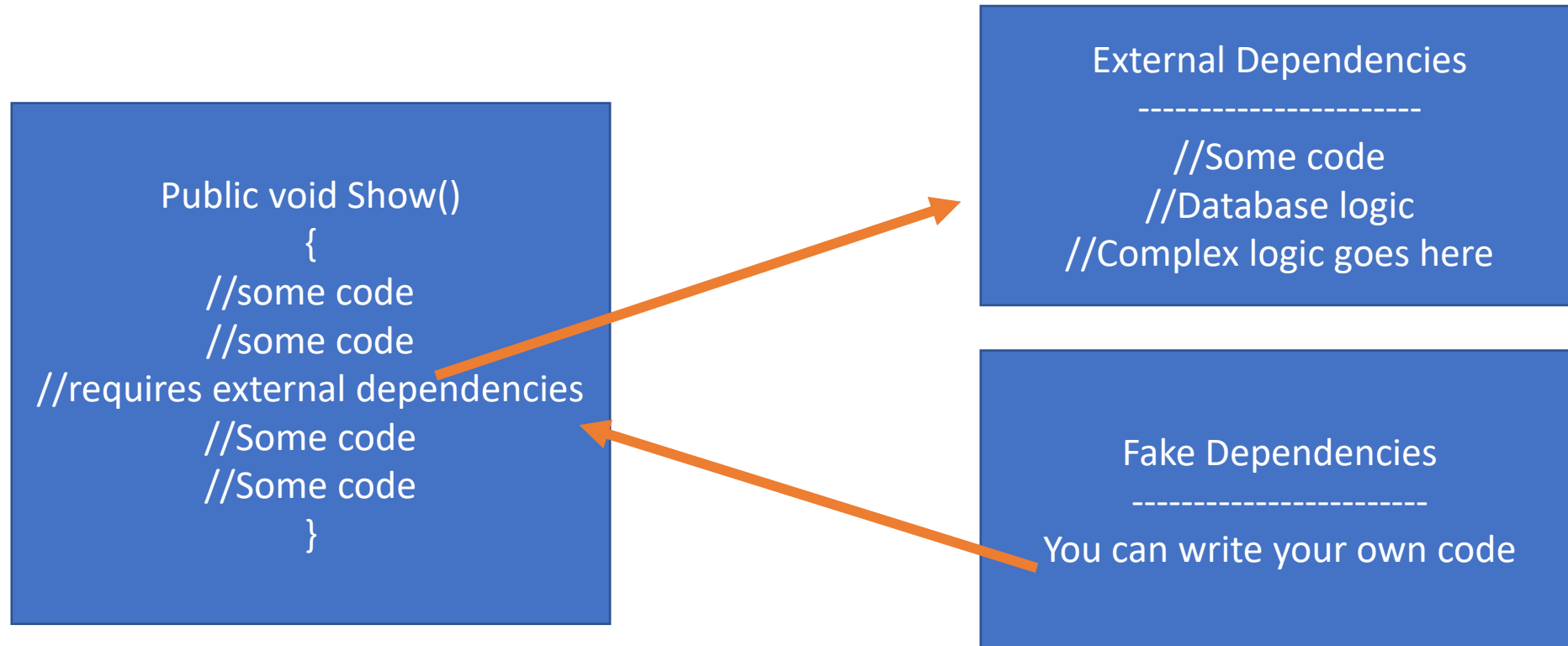
TDD – Test Driven Development

- Step-1 : Write test case before writing fully functional code(Test first development)
- Step-2 : The new test case fails to starts with
- Step-3: Write the code to satisfy and pass the test case

MOQ Framework

- MOQ Means : **Not real / Genuine**
- Using MOQ you write unit tests by **creating fake objects (mocks)** that simulate the behaviour of real objects
- This allows you to isolate the code you're testing and focus on its functionality without worrying about the dependencies on external components.

How MOQ Works?



Advantage of Mocking

- Helps to run Unit tests faster by reducing the execution time
- Helps to write unit tests without depending on external services/components
 - ✓ No need to set up the database to unit test the code.(depends on Data access layer(DAL))
 - ✓ No need to set up the real service to unit test the code depends on service layer
 - ✓ No need to setup the SMTP /email server to unit test the code dependant on SMTP and mail server

MOQ can be achieved in 3 ways

- Stub
- Fake
- MOQ Framework

Stub

- The stub provides a hardcoded result (number) for the method.
- This test doesn't care about the interaction;
- it only ensures the Add method returns a value so the Client can function.

Service class

```
public interface IMath
{
    string Add(int x, int y);
}
internal class Service : IMath
{
    public string Add(int x, int y)
    {
        return "the sum is " + (x + y);
    }
}
```

```
public class Client
{
    IMath math;
    public Client(IMath m)
    {
        math = m;
    }

    public string AddClient(int x, int y)
    {
        return math.Add(x,y);
    }
}
```

Using Stub

```
public class StubCalculatorService : IMath
{
    public string Add(int a, int b)
    {
        return "the stub sum is 5"; // Always returns 5 no
        matter what the inputs are
    }
}
```

```
[Test]
public void Test2()
{
    StubCalculatorService s = new
    StubCalculatorService();

    Client c = new Client(s);

    var res = c.AddClient(10, 20);
    Assert.AreEqual("the fake sum is 30", res);

}
```

Fake

“Fakes are simplified implementations of a class or interface”

Using Fake

```
public class FakeService : IMath
{
    public string Add(int x, int y)
    {
        return "the fake sum is " + (x + y);
    }
}
```

```
[Test]
public void Test1()
{
    FakeService f = new FakeService();

    Client c = new Client(f);

    var res = c.AddClient(10, 20);
    Assert.AreEqual("the fake sum is 30", res);

}
```

MOQ

“Mocks are objects that simulate the behavior of real objects”

MOQ are simplified way to fake object using MOQ package

Using MOQ(Readymade way to fake)

- `var ob = new Mock<IMath>();`
- `ob.Setup(c => c.Add(5,5)).Returns("the moq sum is 10");`
- `Client c = new Client(ob.Object);`
- `var res = c.AddClient(5,5);`
- `Assert.AreEqual("the moq sum is 10", res);`
- `ob.Verify(x => x.Add(5, 5), Times.Once);`

Verify

- used to assert that moq method has been called under certain conditions
- allows you verify the amount of calls
- allows you to verify the parameters passed in

Verify

- the times struct lets you specify invocation amount restriction when verifying
- exactly
- at most
- at least
- between
- once

Setup-Returns

- Allows you to configure what a mocked out method will return
- `Chain.Returns(delegate)` onto a setup method
- Return delegate
 - types of `Func<parameter1,parameter2...returntype>`
 - input : all method arguments
 - output : method output

